ПРАКТИЧЕСКАЯ РАБОТА №3
Символьные строки и форматированный ввод-вывод данных языка Си.

Цель работы: освоить механизмы работы с символьными строками и нуль-терминатором. Изучить библиотеку `<string.h>` для определения длины строк. Научиться применять сложные модификаторы и флаги ширины/точности в функциях `printf()` и `scanf()`. Экспериментально исследовать границы точности вещественных типов и освоить работу с широкими символами для корректного вывода Unicode-строк.

Общее требование: во всех заданиях подключайте `<stdio.h>`, используйте `int main(void)` и завершайте программу оператором `return 0;`. Если в задании используются `<string.h>`, `<float.h>`, `<wchar.h>` или `<locale.h>`, подключите соответствующие заголовочные файлы.

o 3.1. Память строки и ее длина

Объявите символьный массив размером 20 байт и инициализируйте его строкой `"SibSUTIS"`. Выведите размер массива с помощью `sizeof`, длину строки с помощью `strlen()` и количество свободных байт в массиве (разница между размером массива и длиной строки с учетом нуль-терминатора). Объясните, почему `sizeof` и `strlen()` возвращают разные значения. Повторите со строкой `"СибГУТИ"` и сформулируйте вывод после результата выполнения программы.

Таблица 3.1.1 – Тестовые данные

Выходные данные

`SIZEOF: 20`
`STRLEN: 8`
`FREE_BYTES: 11`

3.2. Ловушка ввода строк

Напишите программу, которая считывает два слова с помощью одного вызова `scanf("%s %s", ...)`. Первое слово сохраните в массив `word1` размером 15 байт, второе слово в массив `word2` размером 15 байт. Выведите оба слова в одной строке через дефис, а затем выведите общую длину обоих слов. Объясните, почему `scanf("%s")` останавливается на пробеле и как это влияет на ввод.

Таблица 3.2.1 – Тестовые данные

Входные данные	Выходные данные
`Hello World`	`Hello-World` `TOTAL_LEN: 10`

3.3. Калибровка буфера вывода.

Напишите программу, которая считывает имя пользователя и выводит его на экран в четырех различных форматах для калибровки интерфейса в следующем порядке:

· Имя в двойных кавычках.

· Имя в кавычках, выровненное по правому краю в поле шириной 20 символов.

· Имя в кавычках, выровненное по левому краю в поле шириной 20 символов.

· Имя в кавычках в поле шириной, которая на 5 символа превышает реальную длину имени;

Таблица 3.3.1 – Тестовые данные

Входные данные	Выходные данные

`Altenok`	`"Altenok"`
	`"            Altenok"`
	`"Altenok            "`
	`"     Altenok     "`

3.4. Escape-последовательности и спецсимволы

Объявите строковый литерал, используя escape-последовательности:

`"\x48\x65\x6C\x6C\x6F\t\x57\x6F\x72\x6C\x64\n".`

Выведите этот литерал с помощью `printf()`. Затем выведите длину строки (без учета `\0`) и размер в памяти (с учетом `\0`). В комментарии объясните, какие символы закодированы в шестнадцатеричной форме и что означает `\t`.

Таблица 3.4.1 – Тестовые данные

Выходные данные

`Hello     World`
`STRLEN: 11`
`SIZEOF: 12`

3.5. Символьная арифметика

Напишите программу, которая считывает один символ (строчную букву латинского алфавита) и выполняет следующие преобразования:

1) Выведите код символа в десятичной, шестнадцатеричной и восьмеричной системах.
2) Преобразуйте символ в верхний регистр и выведите результат.
3) Выведите следующий символ в алфавите.
4) Выведите предыдущий символ в алфавите.

Все символьные значения выводите через `char` и спецификатор `%c`.

Таблица 3.5.1 – Тестовые данные

Входные данные	Выходные данные
m	CODE_DEC: 109 CODE_HEX: 6d CODE_OCT: 155 UPPER: M NEXT: n PREV: l

3.6. Динамическая ширина поля

Напишите программу, которая считывает строку и целое число `width`. Выведите строку в поле динамической ширины `width`, выровненную по правому краю. Затем выведите ту же строку в поле ширины `width + 5`, выровненную по левому краю.

Таблица 3.6.1 – Тестовые данные

Входные данные	Выходные данные
Test 10	Test Test

3.7. Константы для строк

Объявите строковую константу с помощью директивы `#define` со значением по типу `"LinuxDistro"`. Объявите константу с помощью модификатора const как массив символов со значением по типу "2026". Выведите директиву и константу, а затем выведите размер строкового литерала с помощью `sizeof` и его длину с помощью `strlen()`. Объясните разницу между размером литерала и его длиной.

Таблица 3.7.1 – Тестовые данные

Выходные данные
DISTRO: LinuxDistro RDATE: 2026 STRLEN: 11 SIZEOF: 12

o 3.8. Работа с расширенным Unicode-стеком.

Напишите программу, которая объявляет две строковые константы: одну в обычном байтовом формате `char` (в кодировке `UTF-8`), а вторую – в формате широких символов `wchar_t` (с префиксом `L""`), содержащие одинаковый кириллический текст (например, `"Сбой матрицы"`). Подключите библиотеки `<wchar.h>`, `<string.h>` и `<locale.h>`. Настройте локаль с помощью

`setlocale(LC_ALL, "")`. Выведите на экран размер каждой строки в байтах и количество символов в каждой строке.

Таблица 3.8.1 – Тестовые данные

Выходные данные
`char  -> sizeof: 24 bytes, strlen: 23 chars` `wchar -> sizeof: 52 bytes, wcslen: 12 chars`

3.9. Дешифрация широкого идентификатора.

Напишите программу, запрашивающую у пользователя ввод широкой строки. Объявите массив типа `wchar_t` достаточного размера и используйте функцию `wscanf()` со спецификатором `%ls` для чтения данных из стандартного потока ввода. После успешного чтения выведите на экран приветственное сообщение в формате:

`"Агент [Позывной] авторизован в системе"`,

используя функцию `wprintf()` и спецификатор `%ls`. Убедитесь, что программа корректно обрабатывает ввод и не выводит "кракозябры", благодаря предварительной инициализации локали.

Таблица 3.9.1 – Тестовые данные

Входные данные	Выходные данные
`Нео`	`Агент Нео авторизован в системе`

3.10. Калибровка интерфейса широких строк.

Напишите программу, которая считывает три широкие строки с помощью `wscanf()`. Выведите эти названия в виде строки телеметрии, используя `wprintf()`. Первый модуль должен быть выровнен по левому краю в поле шириной 15 широких символов. Второй модуль должен быть выровнен по правому краю в поле шириной 20 широких символов. Третий модуль должен быть выведен с жестким ограничением точности ровно в 5 символов. Все три поля разделите символом вертикальной черты.

Таблица 3.10.1 – Тестовые данные

Входные данные	Выходные данные

`Ядро`
`Реактор`
`Инициализация`
	`Ядро                 |                Реактор|Иници`